Design Document for Virtual Router

1. Project Overview

Brief

This project implements a simple user-space virtual router in C for Linux. The application reads interface and static route configuration from JSON files, stores the data in interface and route tables, and performs IPv4 route lookup using longest prefix match (LPM) through CLI commands.

The main focus of the project is routing logic, configuration handling, CLI interaction, and basic routing behaviour simulation using a simple modular design.

Objective

Implement a simple virtual router in C for Linux environments that performs:

- Static route management
- Longest Prefix Match (LPM) route lookup
- CLI-based route/interface inspection
- Route lookup explanation
- The application runs completely in user-space on Linux and does not require root privileges.
- Runtime route add/modify/delete support
- Runtime interface up/down handling
- Interface operational/admin status management
- Route active/inactive state handling
- Interface statistics display support
- Explain-lookup support

2. High Level Design (HLD)

MAIN COMPONENTS:

1. **JSON Configuration Loader** : json config loader will do following actions
   - Open interfaces.json file in rb mode
   - Read and parse the file
   - Store interface configuration in predefined interface_table array including:
     - Interface name
     - IP address
     - Prefix length
     - Administrative state
     - Operational state
     - RX/TX packet statistics
     - RX/TX byte statistics
   - Connected Route Handling:
     - Calculate connected network using:
       network = ip_address & subnet_mask
   - Populate route_table with connected routes
   - Update route type as Connected
   - Update route active/inactive status based on interface operational state

- Static route handling:
  The static route loader performs the following actions:
  - Opens static_routes.json in rb mode
  - Reads and parses static route entries
  - Traverses route_table[]
  - Identifies unfilled or reusable route table entries
  - Stores parsed route information into the global route table

  The following route information is supported:
  - Destination network
  - Prefix length
  - Next hop
  - Preference value
  - Egress interface (optional)
  - DIRECT next-hop handling

  Static routes are inserted with:
  - Route type = Static
  - Active route state
  - Preference handling support

  Duplicate route insertion handling is also supported during route population.

2. **CLI Module** : The CLI module provides runtime interaction with the virtual router application.

Supported Features
1. Interface display commands
2. Interface statistics display
3. Route table display
4. Runtime route add/delete/modify operations
5. Runtime interface up/down handling
6. IPv4 lookup
7. Explain-lookup support
8. Help command support
9. Command validation handling
10. Command history navigation using arrow keys

CLI Validation Handling

The CLI validates and handles:
1. Invalid IPv4 inputs
2. Missing command arguments
3. Invalid prefix values
4. Unknown commands
5. Invalid route operations

The exit command terminates the CLI loop and exits the application.

3. **Route Table Manager** : The Route Table Manager acts as an interface layer between the CLI module and the LPM engine.
   Responsibilities include:
   1. Runtime route management
   2. Route activation/deactivation
   3. Interface status synchronization
   4. Route lookup request handling
   5. Explain-lookup request handling
   6. Route and interface display handling
   7. Route preference handling
   8. Coordination between CLI and routing logic

The Route Table Manager also coordinates formatting and printing operations through the print utility layer.

4. **Longest Prefix Match (LPM) Engine** : LPM engine will perform longest prefix match calculation and return best matching route info to Route Table Manager.

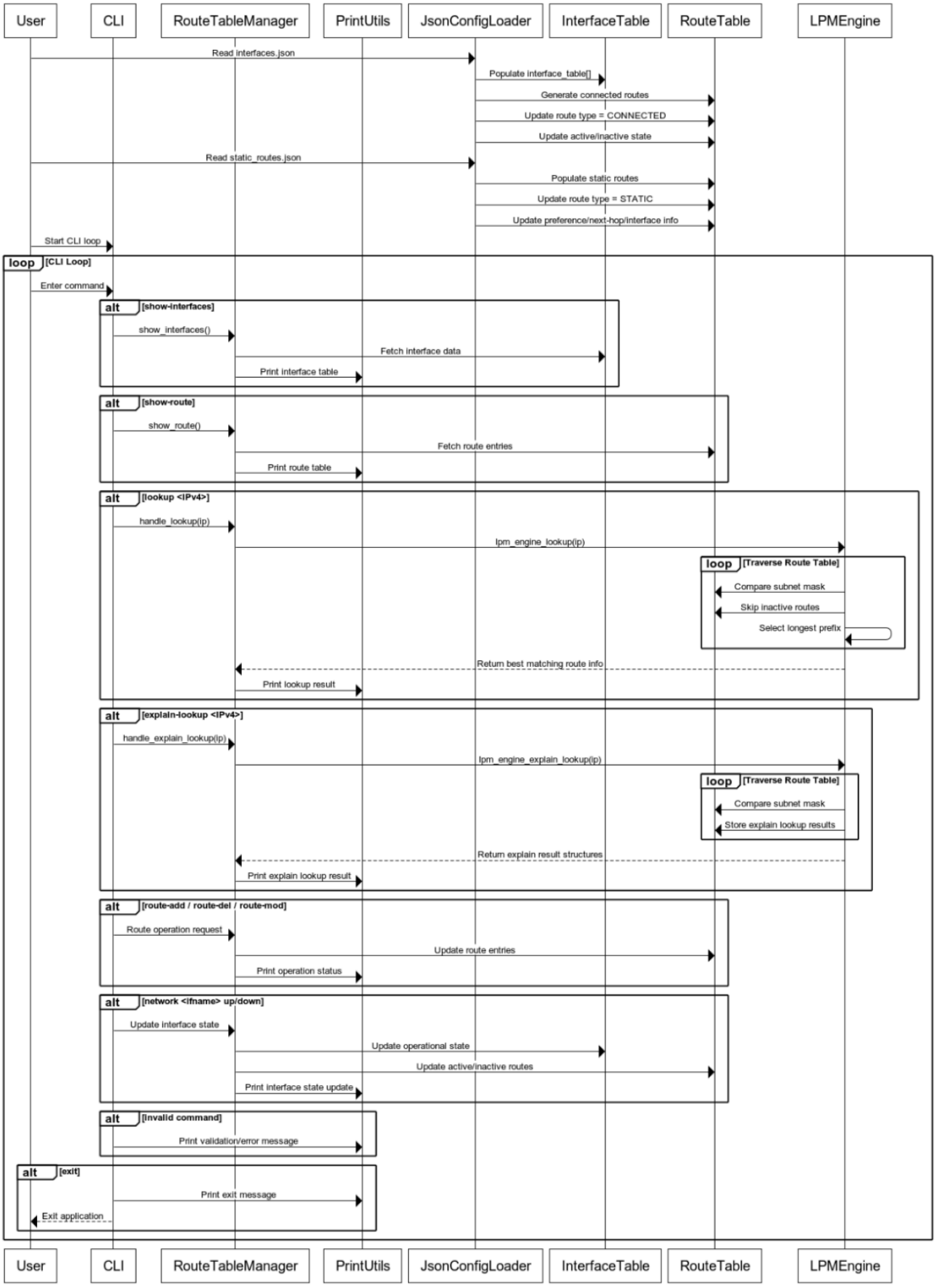
   LPM engine enhancements:
   - Skips inactive routes during lookup
   - Supports explain-lookup flow
   - Returns best matching route index
   - Uses linear search based longest prefix match
   - Supports default route fallback

   Lookup Steps:
   1. Receive IPv4 address in integer format
   2. Traverse the route table
   3. Apply subnet mask comparison
   4. Compare matching prefix lengths
   5. Select the route with the longest matching prefix
   6. Return best matching route information

HIGH LEVEL FLOW

1. Read interface configuration from interfaces.json
2. Store interface information into interface_table[]
3. Generate connected routes and populate route_table[]
4. Read static routes from static_routes.json
5. Populate static route entries into route_table[]
6. Start CLI loop
7. Accept and parse user commands
8. Forward interface and route display requests to Route Table Manager
9. Forward lookup and explain-lookup requests to LPM engine
10. Perform longest prefix match calculation
11. Return matching route information to Route Table Manager
12. Print formatted output through CLI/print utility layer.

Virtual Router CLI and LPM Flow



3. Project Structure

4. Data Structures and Enums

Macros:

```
#define MAX_ROUTES      128
#define MAX_INTERFACES  16
#define INTERFACE_NAME_LEN 32
```

Interface Table Structure:

```
typedef struct interface_t
{
    char name[INTERFACE_NAME_LEN];
    uint32_t ip_addr;
    uint8_t prefix_len;
    uint8_t admin_up;
    uint8_t is_up; /* operational status: 1 for up, 0 for down */
    uint64_t rx_packets;
    uint64_t tx_packets;
    uint64_t rx_bytes;
    uint64_t tx_bytes;
} _interface_t;
```

Route Table Structure:

```
typedef struct route_t
{
    uint32_t network;
    uint8_t prefix_len;
    uint32_t next_hop;
    char interface[INTERFACE_NAME_LEN];
    _route_type_t route_type;
    bool is_active; /* used to mark routes
                       as active/inactive based on interface status
                       or other criteria */
    uint16_t preference; /* lower value means higher preference, used for tie-breaking */
} _route_t;
```

Route Table Enums:

```
typedef enum route_interface_status_t
{
    INACTIVE_ROUTE = 0,
    ACTIVE_ROUTE
} _route_interface_status_t;
typedef enum route_type_t
{
    ROUTE_TYPE_CONNECTED = 0,
    ROUTE_TYPE_STATIC
} _route_type_t;
```

ipm_engine_explain Structure:

```
typedef struct explain_t
{
    int idx;
    uint32_t matched_prefix;
    int prefix_len;
    char interface[16];
    uint8_t route_type;
    uint8_t is_active;
    int preference;
    u_int32_t next_hop;
} _explain_t;
```

5. JSON Configuration

The virtual router reads configuration data from JSON files during initialisation.

interfaces.json Contains:

- Interface name
- IP_Prefix
- Admin State
- Operational State
- Rx Packets, Tx Packets
- Rx Bytes, Tx Bytes

Example:


```
{
   "name": "eth0",
   "ip_prefix": "10.0.0.1/24",
   "admin_state": "up",
   "oper_state": "up",
   "rx_packets": 1200,
   "tx_packets": 980,
   "rx_bytes": 240000,
   "tx_bytes": 180500
 },
{
   "name": "eth1",
   "ip_prefix": "192.168.1.1/24",
   "admin_state": "up",
   "oper_state": "up",
   "rx_packets": 800,
   "tx_packets": 650,
   "rx_bytes": 125000,
   "tx_bytes": 110200
 }
```


static_routes.json Contains:

- Prefix
- Prefix length
- Next hop
- Engress interface
- Preference

Example:


```
 {
   "prefix": "172.16.0.0/16",
   "next_hop_ip": "192.168.1.254",
   "preference": 10

 },
 {
   "prefix": "172.16.10.0/24",
   "egress_interface": "eth0",
   "preference": 20
 }
```

6. CLI Commands

The following CLI commands have been implemented in the virtual router application:

Command	Description
show-interfaces	Display all interface information
show-interface <eth>	Display information for a specific interface
show-interface-stats <eth>	Display statistics for a specific interface
show-all-interface-stats	Display statistics for all interfaces
show-route / show-routes	Display route table entries
network <eth> down|up	Update interface operational state
route-add <net> <prefix> <nexthop> <interface>	Add a new route
route-del <net> <prefix> <nexthop> <interface>	Delete an existing route
route-mod <net> <prefix> <nexthop> <interface> <preference>	Modify an existing route
lookup <IPv4>	Perform longest prefix route lookup
explain-lookup <IPv4>	Display route lookup match flow
help / h / ?	Display CLI help message
exit / quit / q	Exit the CLI

CLI Usage Example:

7. Assumptions/Limitations

- No actual packet forwarding
- No recursive next-hop resolution
- Linear search based LPM implementation
- IPv4 only
- No kernel integration
- No multithreading

8. Validation and Testing

Test Case Name	Details	Status
test_valid_show_interfaces()	Validate show-interfaces, show-interface <eth>, show-interface-stats <eth>, and show-all-interface-stats commands display correct interface details and statistics.	Completed
test_valid_show_route()	Validate show-route / show-routes command prints all configured routes with correct network, prefix, next hop, interface, route type, status, and preference.	Completed
test_invalid_command_handling()	Verify CLI properly handles unsupported or unknown commands and prints validation/error messages.	Completed
test_missing_arg_validation()	Validate detection of missing CLI arguments for commands like route-add, route-mod, route-del, lookup, and network.	Completed
test_invalid_ipv4_and_prefix()	Verify invalid IPv4 addresses and invalid prefix lengths (>32 or malformed values) are rejected correctly.	Completed
test_route_add_and_duplicate()	Validate runtime route addition and ensure duplicate routes are detected/prevented properly.	Completed
test_route_mod_and_del()	Validate runtime route modification and deletion functionality including route lookup by network/prefix.	Completed
test_lookup_exact_and_default()	Verify LPM engine selects exact matching route first and falls back to default route when no better match exists.	Completed
test_explain_lookup_validation()	Validate explain-lookup command output including matched prefixes, selected route, and traversal details.	Completed
test_network_up_down_and_interface_not_found()	Verify interface operational/admin state updates using network <eth> up/down and validate handling of invalid interface names.	Completed
test_empty_tables_and_boundary_handling()	Validate behavior when interface table or route table is empty and test maximum table boundary conditions.	Completed
test_help_and_exit_and_whitespace()	Validate help, h, ?, exit, quit, and whitespace-only input handling.	Completed
test_cli_unknown_and_whitespace_tab_handling()	Verify CLI correctly handles unknown commands, extra spaces, tab-separated inputs, and malformed command formatting.	Completed
test_add_and_delete_routes()	Validate adding and deleting multiple routes dynamically and confirm route table consistency after operations.	Completed
test_lpm_longest_prefix_choice()	Verify longest prefix match logic selects the most specific route among multiple matching entries.	Completed
test_default_route_behavior()	Validate default route (0.0.0.0/0) selection when no specific route match exists.	Completed
test_invalid_and_boundary_cases()	Validate edge cases including route table full conditions, invalid interfaces, inactive routes, null values, and boundary prefix lengths (0 and 32).	Completed

9. Alternate Design Considerations

The current implementation uses a simple modular design focused on readability, debugging simplicity, and ease of enhancement. During development, multiple alternate approaches were considered.

1. Trie-Based LPM Lookup

Current Design:
- Linear search across route_table array

Alternate Design:
- Trie / Patricia Trie based lookup engine

Reason Not Selected:
- Route table size is small
- Linear search simplifies debugging and validation
- Reduced implementation complexity for assignment scope

Potential Benefit:
- Faster lookup for large route scale

2. Dynamic Memory Allocation

Current Design:
- Static global arrays for interface_table and route_table

Alternate Design:
- Dynamically allocated route/interface entries using linked lists or trees

Reason Not Selected:
- Static arrays simplify memory management
- Easier debugging and deterministic behavior
- Avoids fragmentation and cleanup complexity

3. Packet Forwarding Simulation

Current Design:
- Control-plane style routing simulation only

Alternate Design:
- Simulated packet receive/forward pipeline using raw sockets or PCAP

Reason Not Selected:
- Out of current project scope
- Main focus was route management and lookup logic

4. Recursive Next-Hop Resolution

Current Design:
- Direct next-hop lookup only

Alternate Design:
- Recursive route resolution similar to production routers

Reason Not Selected:
- Simplified routing workflow
- Reduced implementation complexity

5. Route Synchronization Strategy

Current Design:
- Route active/inactive status updated based on interface state

Alternate Design:
- Dynamic recalculation/removal of routes during interface state changes

Reason Not Selected:
- Status flag based design provides simpler runtime state management
- Faster interface up/down handling
- Avoids repeated route creation/deletion

10. AI Usage Summary

Complete AI interaction history is documented separately in AI_USAGE.md.

11. Future Enhancements

- Recursive next-hop resolution
- Trie based LPM optimization
- Dynamic routing protocols
- Packet forwarding simulation
- IPv6 support
- Persistent runtime configuration save
- Netlink/socket integration

12. Tools Used

Purpose	Purpose
Visual Studio Code	Source code development and debugging
OpenAI Codex	AI-assisted coding support and code suggestions
ChatGPT	Design document template, brief preparation, and technical discussions
WebSequenceDiagrams	Sequence diagram creation
GNU Make	Build automation
GCC	C compilation

13. Reference Study Materials

The following references were used to understand routing concepts, CLI flow, and implementation approach during development:

* Linux Routing Fundamentals Video Reference :
https://youtu.be/svEvF6nK36o?si=-f1ykukXaZKifTJ0

14. Key Learnings

- IPv4 routing fundamentals
- LPM implementation
- CLI parsing
- JSON parsing in C
- Route/interface state synchronization
- Runtime configuration handling
- Debugging and validation